

Understanding Email and How It Works

Email, which stands for electronic mail, is essentially the digital version of traditional postal mail. You write an email message, specify the recipient's email address and your own email address, click send, and the email travels through the internet to reach the recipient's inbox. Just like you cannot deliver a letter directly to someone's house without going through the postal system, you cannot send an email directly from your computer to another person's computer without going through an email server system.

Email Components

From: you@gmail.com	← Sender address
To: friend@gmail.com	← Recipient address
Subject: Welcome!	← Email title
Body: Hello friend!	← Email content

[SMTP](#), or Simple Mail Transfer Protocol, is the standard internet protocol for sending electronic mail. Using SMTP directly gives you full control over the process, but it also comes with more responsibility.

Python's standard library includes a powerful module called `smtplib`, which allows us to communicate directly with an SMTP server.

Why Can't I Send Email Directly?

The reason you cannot send email directly from your computer is because your computer lacks several critical components that are required for email delivery. First, your computer has a private IP address that the internet cannot reach directly. Second, email servers around the world do not know or trust your computer, so they would block any emails coming from it as potential spam. Third, proper email delivery requires specific DNS records like MX records, SPF, and DKIM that verify your identity as a legitimate sender, and these are complex to set up on

your own. Fourth, most internet service providers block port 25, which is the standard port for sending emails, to prevent spam. Finally, there is no authentication mechanism to prove that the email is really from you and not from someone pretending to be you. All of these problems are solved by using an SMTP server, which acts like a post office for your emails.

Why Direct Sending Doesn't Work

- No Public IP Address

Your computer has private IP (192.168.x.x)

Internet can't reach you directly

- No Reputation

Email servers don't know your computer

They'll block you as spam

- No DNS Records

MX records, SPF, DKIM required

Complex to setup

- Blocked by ISPs

Port 25 (SMTP) blocked by most ISPs

Prevents spam

- No Authentication

How does recipient know it's really you?

Need verified identity

What is SMTP?

SMTP = Simple Mail Transfer Protocol

PROTOCOL COMPARISON

HTTP → Transfer web pages (websites)

SMTP → Transfer emails

FTP → Transfer files

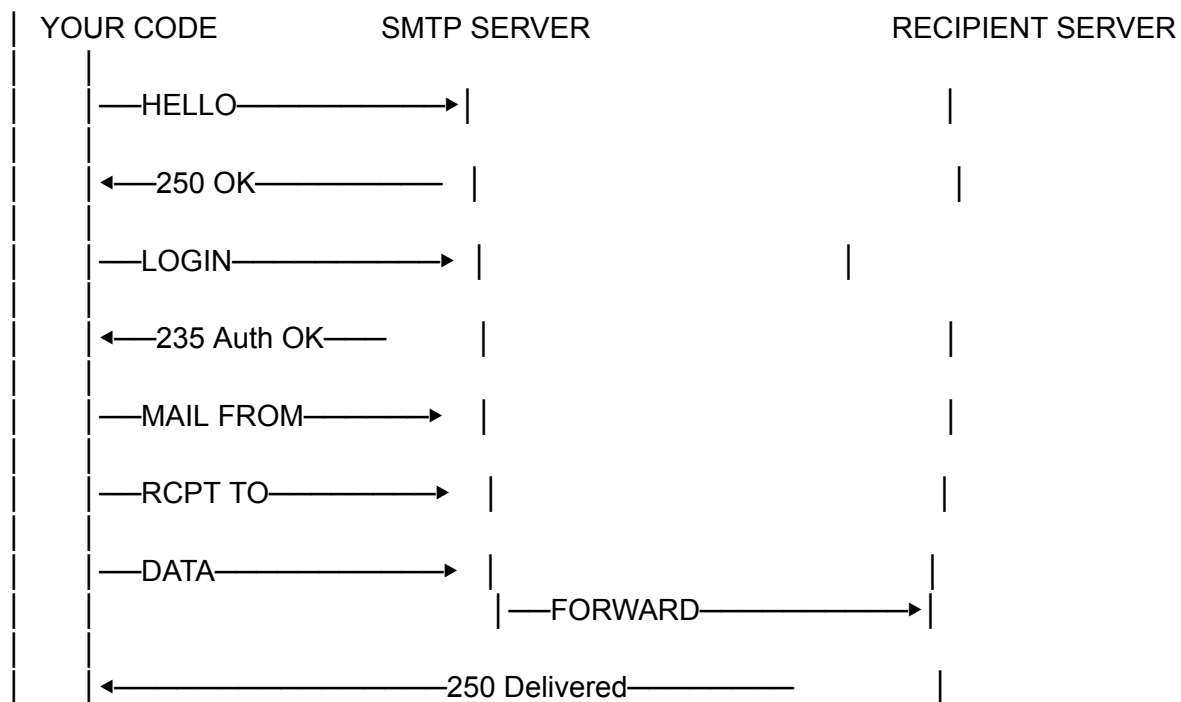
TCP/IP → Transfer any data on internet

SMTP is the RULES for sending emails

Just like HTTP is the RULES for websites

SMTP stands for Simple Mail Transfer Protocol, and it is simply the set of rules that govern how emails are sent over the internet. Just like HTTP is the protocol for transferring web pages and FTP is the protocol for transferring files, SMTP is the protocol specifically designed for transferring emails. When you send an email, your computer connects to an SMTP server, authenticates itself with a username and password, and then hands over the email to the server. The SMTP server then takes responsibility for delivering that email to the recipient's email server. This is why you need an SMTP server, because it has all the infrastructure, reputation, and authentication that your personal computer lacks.

SMTP Conversation (Simplified)



4. Understanding Email Configuration

MAIL_USERNAME

This is your email address. For Gmail, use the full address with @gmail.com. The SMTP server needs to know who is sending.

Example:

MAIL_USERNAME = john@gmail.com

MAIL_PASSWORD

This is your email password. For Gmail, use App Password not regular password. App Password is 16 characters. It is more secure for applications.

Example:

MAIL_PASSWORD = abcdefghijklmnop

MAIL_FROM

This is the sender email address. The recipient sees this as "From". For Gmail, it must match MAIL_USERNAME.

Example:

MAIL_FROM = john@gmail.com

MAIL_PORT

This is the port number for SMTP server. Port 587 uses TLS encryption. Port 465 uses older SSL. Port 25 is often blocked.

Example:

MAIL_PORT = 587

MAIL_SERVER

This is the SMTP server address. Different providers have different servers. Gmail uses smtp.gmail.com.

Example:

MAIL_SERVER = smtp.gmail.com

5. Why Username and Password Are Needed

Authentication Process

Your code connects to smtp.gmail.com. The server asks "Who are you?". Your code sends MAIL_USERNAME.

Verification Step

The server asks "Prove it!". Your code sends MAIL_PASSWORD. The server verifies the credentials. If valid, email sending is allowed.

Why This Matters

This prevents spammers from using Gmail servers. Only authorized users can send emails. This keeps the email system secure.

6. How Email Travels Step by Step

Step 1: Create Email

Your Python code calls `send_credentials_email` function. You provide recipient email, username, password, and organization name.

Step 2: Connect to SMTP Server

Your code connects to `smtp.gmail.com` on port 587. It sends `MAIL_USERNAME` and `MAIL_PASSWORD`. Gmail verifies these credentials.

Step 3: Server Accepts Email

Gmail SMTP server accepts your email. It adds the email to its delivery queue. It checks your sender reputation.

Step 4: Find Recipient Server

Server performs DNS lookup. It finds the MX record for recipient's domain. It gets the address of recipient's email server.

Step 5: Deliver Email

Server connects to recipient's email server. It transfers the email. The recipient's server runs spam filters.

Step 6: Recipient Receives

Email appears in recipient's inbox. They open and read your email. They see the login credentials you sent.

Step to implement Email notification:

Step 1: import these thing from .env

```
SMTP_HOST = os.getenv("SMTP_HOST")
SMTP_PORT = int(os.getenv("SMTP_PORT"))
SMTP_USER = os.getenv("SMTP_USER")
SMTP_PASSWORD = os.getenv("SMTP_PASSWORD")
EMAIL_FROM = os.getenv("EMAIL_FROM")
```

Here the email is personal email and pwd is personal email password or set it by set app password in google account setting >security>app password.

Step2: create an email template, content you want to send in email for this:

Create a template /email.html file and set the styling of your email

```
<h3>🔑 Your Login Credentials:</h3>
    <table>
      <tr>
        <td>Username:</td>
        <td>{{ username }}</td>
      </tr>
      <tr>
        <td>Email:</td>
        <td>{{ user_email }}</td>
      </tr>
    </table>
```

```

        <td>Password:</td>
        <td style="color: #dc3545; font-family:
monospace;">{{ password }}</td>
    </tr>
    <tr>
        <td>Organization:</td>
        <td>{{ org_name }}</td>
    </tr>
</table>

```

Step3: create a utils/email_service.py in which you load html file and send email by smtp server

```

BASE_DIR = Path(__file__).resolve().parent.parent
TEMPLATE_DIR = BASE_DIR / "template"

EMAIL_TEMPLATE = TEMPLATE_DIR / "email.html"

```

```

env = Environment(
    loader=FileSystemLoader(TEMPLATE_DIR),
    autoescape=select_autoescape(['html', 'xml'])
)

def send_credentials_email(to: str, username: str, password: str,
org_name: str):

    try:
        # Load template
        template = env.get_template("email.html")

        # Render template with variables
        body = template.render(
            username=username,
            user_email=to,
            password=password,
            org_name=org_name
        )

        # Create email message

```

```

message = MIMEMultipart()
message["From"] = EMAIL_FROM
message["To"] = to
message["Subject"] = f"Welcome to {org_name} - Your Account
Credentials"

# Attach HTML body
message.attach(MIMEText(body, "html"))

# Send email
with smtplib.SMTP(SMTP_HOST, SMTP_PORT) as server:
    server.starttls()
    server.login(SMTP_USER, SMTP_PASSWORD)
    server.sendmail(EMAIL_FROM, to, message.as_string())

logger.info(f"Email sent successfully to {to}")
return True

except Exception as e:
    logger.error(f" Error sending email to {to}: {str(e)}")
    return False

```

Step4: update user handler and user router

```

try:
    send_credentials_email(
        to=u.email,
        username=u.username,
        password=u.password,
        org_name=org.name
    )
    logger.info(f"📧 Email sent to {u.email}")
except Exception as e:
    logger.warning(f"Email failed to send to {u.email}:
{str(e)}")

```